

题型：选择题 (10个) 10分

填空题 (10个) 10分

判断题 (10个) 10分

综合题 (8个) 40分

应用题 (12个) 30分

复习内容

第一章

1. 程序设计语言的发展阶段 (机器语言、汇编语言、高级语言)

机器语言 —— 汇编语言 —— 高级语言

目标程序

源程序

编译的本质即翻译：将高级语言 ——> 机器语言或汇编语言

源程序 ——> 目标程序

2. 执行高级语言的途径以及二者的主要区别

1.2 计算机执行用高级语言编写的程序有哪些途径？它们之间的主要区别是什么？

有两个途径：解释与编译。

编译方式不同。

编译：是将高级语言全部翻译成机器代码程序，然后再整体运行

解释：每读入一条源程序就解释执行，即边解释边执行

3. 编译的定义，源语言，目标语言

高级语言

机器语言或汇编语言

1.1 完成下列选择题:

(1) 下列叙述中正确的是 A。 ^{语言} 转为 目标程序 (汇编程序或机器语言)

- A. 编译程序是将高级语言程序翻译成等价的机器语言程序的程序
B. 机器语言因其使用过于困难, 所以现在计算机根本不使用机器语言
C. 汇编语言是计算机唯一能够直接识别并接受的语言
D. 高级语言接近人们的自然语言, 但其依赖具体机器的特性是无法改变的

(2) 将编译程序分成若干个“遍”是为了 B。

- A. 提高编译程序的执行效率
B. 使编译程序的结构更加清晰
C. 利用有限的机器内存并提高机器的执行效率
D. 利用有限的机器内存但降低了机器的执行效率

(3) 构造编译程序应掌握 D。

- A. 源程序
B. 目标语言
C. 编译方法
D. 以上三项

(4) 编译程序绝大多数时间花在 B, D 上。 ^{编译各个阶段都涉及构造、查有关表格。}

- A. 出错处理
B. 词法分析
C. 目标代码生成
D. 表格管理

(5) 编译程序是对 B, D

- A. 汇编程序的翻译
B. 高级语言程序的解释执行
C. 机器语言的执行
D. 高级语言的翻译

分多遍完成编译过程可以使整个编译程序的逻辑结构更清晰

4. 编译程序的阶段 (5阶段)

词法分析

把字符(从左到右)转为单词符号

语法分析

把单词符号分解为各类语法单位

语义分析和中间代码生成

① 进行初步翻译 ② 收集标识符的属性信息

优化

对前阶段产生的中间代码进行等价变换或改造

目标代码生成

把中间代码变成特定的机器语言或汇编语言

8. 编译程序的开发: 自编译、交叉编译、自展和移植等技术的实现

9. 构造编译程序所应具备的知识内容: 源程序、目标程序、编译方法

第二章 词法分析 (任务) 从左至右逐个字符也对源程序进行扫描

1. 执行词法分析的程序称为词法分析程序，又称词法分析器或扫描器

功能：输入源程序，输出单词符号

两种处理结构

把词法分析程序作为主程序

把词法分析程序作为语法分析程序调用的子程序

(单词符号)

只执行一次是最好的 ×

2. 输出形式：(单词种别，单词自身的值)

3. 正规表达式与正规集

简称正规式，它表示的集合即为正规集

$$(L(a|b))^* = (L(a) \cup L(b))^* = (\{a\} \cup \{b\})^* = \{a, b\}^*$$

eg: $(ab)^*a$ 与 $a(ba)^*$ 是否等价

$$(ab)^*a = \{\epsilon, ab, abab, \dots\}a = \{a, aba, ababa, \dots\}$$

$$a(ba)^* = a\{\epsilon, ba, baba, \dots\} = \{a, aba, ababa, \dots\}$$

克林闭包 $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \dots$ (带*的) 即包含空串 ϵ

$$\text{eg: } \{a, b, c\}^* = \{\Sigma, a, b, c, aa, ab, ac, ba, bb, bc, ca, cb, cc, \dots\}$$

$\{a, b, c\} \times \{a, b, c\}$

正闭包 $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \dots$ (带+的) 不包含空串 ϵ

4. 有限自动机(FA) { 确定有限自动机(DFA) 非确定有限自动机(NFA)

1) 一个确定的有限自动机 M_d (记为 DFA M_d) 是一个五元组 $M_d = (S, \Sigma, f, s_0, Z)$

S : 有限状态集

Σ : 有穷输入字母表

f : 从 $S \times \Sigma$ 到 S 的单值映射

从 $S \times \Sigma^*$ 到 S 的子集映射

s_0 : 唯一的一个初态

非空初态集

Z : 终态集

例 2.4 假定 DFA $M_d = (\{s_0, s_1, s_2\}, \{a, b\}, f, s_0, \{s_2\})$, 且有:

$f(s_0, a) = s_1$

$f(s_0, b) = s_2$

$f(s_1, a) = s_1$

$f(s_1, b) = s_2$

$f(s_2, a) = s_2$

$f(s_2, b) = s_1$

试给出 DFA M_d 的状态转换图与状态转换矩阵。

[解答] DFA M_d 的状态转换图见图 2-8, 状态转换矩阵见表 2.2。

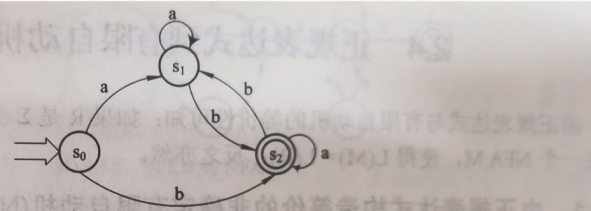


图 2-8 例 2.4 的 DFA M_d 状态转换图

表 2.2 状态转换矩阵

状态 \ 字符	a	b
s_0	s_1	s_2
s_1	s_1	s_2
s_2	s_2	s_1

例 2.5 假定 NFA $M_n = (\{s_0, s_1, s_2\}, \{a, b\}, f, \{s_0, s_2\}, \{s_1\})$, 且有:

$f(s_0, a) = \{s_2\}$

$f(s_0, b) = \{s_0, s_1\}$

$f(s_1, a) = \Phi$

$f(s_1, b) = \{s_2\}$

$f(s_2, a) = \Phi$

$f(s_2, b) = \{s_1\}$

试给出 NFA M_n 的状态转换图与状态转换矩阵。

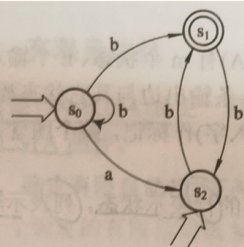


图 2-9 例 2.5 的 NFA M_n 的状态转换图

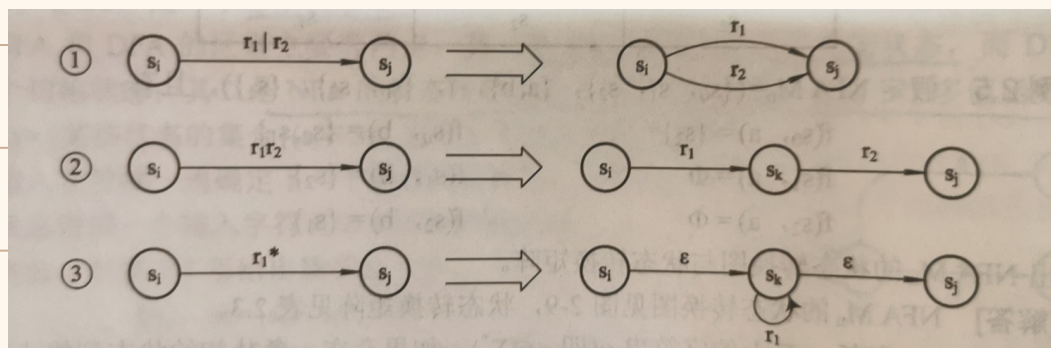
表 2.3 状态转换矩阵

状态 \ 字	a	b
s_0	$\{s_2\}$	$\{s_0, s_1\}$
s_1	Φ	$\{s_2\}$
s_2	Φ	$\{s_1\}$

6. 由正规表达式构造有限自动机

正规式 $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ DFA化简

1) 转换规则



例 2.6 对给定正规表达式 $b^*(d|ad)(b|ab)^+$ 构造其 NFA M。

[解答] 先用 $R^+ = RR^*$ 改造题设的正规表达式为 $b^*(d|ad)(b|ab)(b|ab)^*$ ，然后构造其 NFA M，如图 2-12 所示。

$$R^+ = RR^*$$

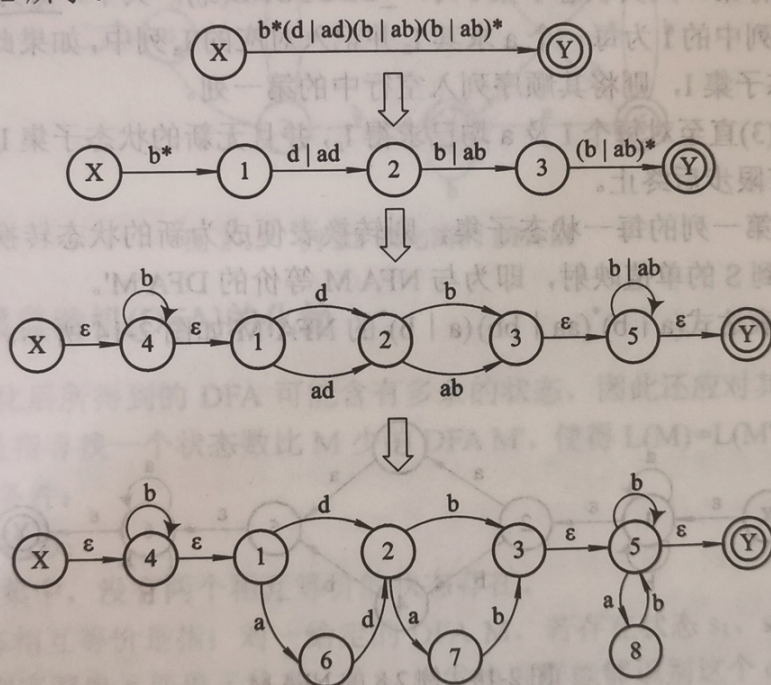


图 2-12 例 2.6 的 NFA M

7. 一个 Lex 源程序由用 "%%" 分隔的三部分组成：辅助定义式、识别规则、用户子程序

2.1 完成下列选择题：

(1) 词法分析所依据的是 B.

- A. 语义规则
- C. 语法规则

(2) 词法分析器的输入是 B.

- A. 单词符号串
- C. 语法单位

(3) 词法分析器的输出是 B.

- A. 单词的种别编码
- C. 单词在符号表中的位置

B. 构词规则

D. 等价变换规则

B. 源程序

D. 目标程序

B. 单词的种别编码和自身的值

D. 单词自身值

(4) 状态转换图(见图 2-1)接受的字集为 D.

- A. 以 0 开头的二进制数组成的集合
- B. 以 0 结尾的二进制数组成的集合
- C. 含奇数个 0 的二进制数组成的集合
- D. 含偶数个 0 的二进制数组成的集合**

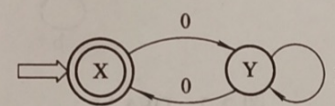


图 2-1 习题 2.1 的 DFA M

- (5) 对于任一给定的 NFA M , B. 一个 DFA M' , 使 $L(M)=L(M')$ 。
 A. 一定不存在 B. 一定存在 C. 可能存在 D. 可能不存在
- (6) DFA 适用于 C.。
 A. 定理证明 B. 语法分析 C. 词法分析 D. 语义加工
- (7) 下面用正规表达式描述词法的论述中, 不正确的是 C.。
 A. 词法规则简单, 采用正规表达式已足以描述
 B. 正规表达式的表示比上下文无关文法更加简洁、直观和易于理解
 C. 正规表达式描述能力强于上下文无关文法
 D. 有限自动机的构造比下推自动机简单且分析效率高
- (8) 与 $(alb)^*(alb)$ 等价的正规式是 A.。
 A. $(alb)(alb)^*$ B. a^*lb^* C. $(ab)^*(alb)^*$ D. $(alb)^*$
- (9) 在状态转换图的实现中, B. 一般对应一个循环语句。
 A. 不含回路的分叉结点 B. 含回路的状态结点
 C. 终态结点 D. A~C 都不是
- (10) 已知 DFA $M_d = (\{s_0, s_1, s_2\}, \{a, b\}, f, s_0, \{s_2\})$, 且有:

$$\begin{aligned} f(s_0, a) &= s_1 & f(s_1, a) &= s_2 \\ f(s_2, a) &= s_2 & f(s_2, b) &= s_2 \end{aligned}$$

则该 DFA M 所能接受的语言可以用正规表达式表示为 B.。

- A. $(a|b)^*$ B. $aa(a|b)^*$ C. $(a|b)^*aa$ D. $a(a|b)^*a$

2.3 设 $M=(\{x,y\}, \{a,b\}, f, x, \{y\})$ 为一非确定的有限自动机, 其中 f 定义如下:

$$\begin{aligned} f(x,a) &= \{x,y\} & f(x,b) &= \{y\} \\ f(y,a) &= \Phi & f(y,b) &= \{x,y\} \end{aligned}$$

试构造相应的确定有限自动机 M' 。

【解答】 对照自动机的定义 $M=(S, \Sigma, f, s_0, Z)$, 由 f 的定义可知 $f(x,a)$ 、 $f(y,b)$ 均函数, 因此 M 是一非确定有限自动机。

先画出 NFA M 相应的状态图, 如图 2-3 所示。

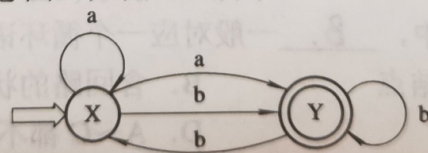


图 2-3 习题 2.3 的 NFA M

用子集法构造状态转换矩阵, 如表 2-1 所示。

表 2-1 状态转换矩阵

I	I_a	I_b
$\{x\}$	$\{x,y\}$	$\{y\}$
$\{y\}$	—	$\{x,y\}$
$\{x,y\}$	$\{x,y\}$	$\{x,y\}$

将转换矩阵中的所有子集重新命名, 形成表 2-2 所示的状态转换矩阵, 即得到 $M'=(\{0,1,2\}, \{a,b\}, f, 0, \{1,2\})$, 其状态转换图如图 2-4 所示。

表 2-2 重命名后的状态转换矩阵

f	字符	a	b
状态			
0		2	1
1		—	2
2		2	2

将图 2-4 所示的 DFA M' 最小化。首先，将 M' 的状态分成终态组 $\{1,2\}$ 与非终态组 $\{0\}$ 。其次，考察 $\{1,2\}$ 。由于 $\{1,2\}_a = \{1,2\}$, $\{1,2\}_b = \{2\} \subset \{1,2\}$ ，因此不再将其划分了，也即整个划分只有两组： $\{0\}$ 和 $\{1,2\}$ 。令状态 1 代表 $\{1,2\}$ ，即把原来到达 2 的弧都导向 1，并删除状态 2。最后，得到如图 2-5 所示的化简了的 DFA M' 。

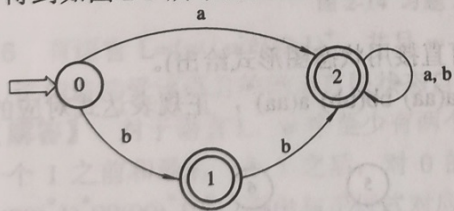


图 2-4 习题 2.3 的 DFA M'

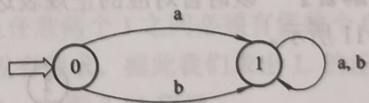


图 2-5 图 2-3 化简后的 DFA M'

例 2.8 正规表达式 $(a|b)^*(aa|bb)(a|b)^*$ 的 NFA M 如图 2-14 所示，试将其确定化为 DFA M' 。

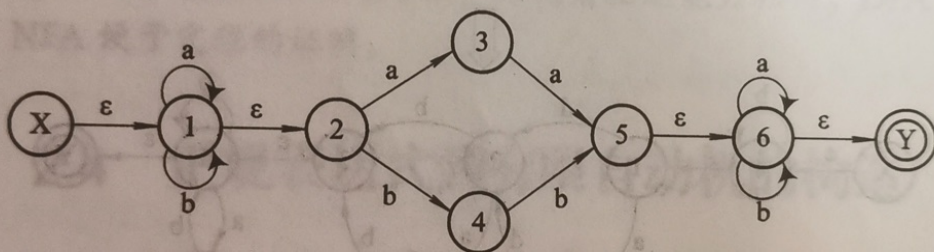
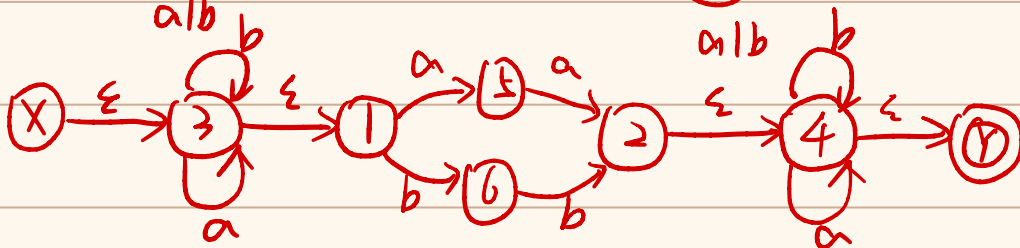
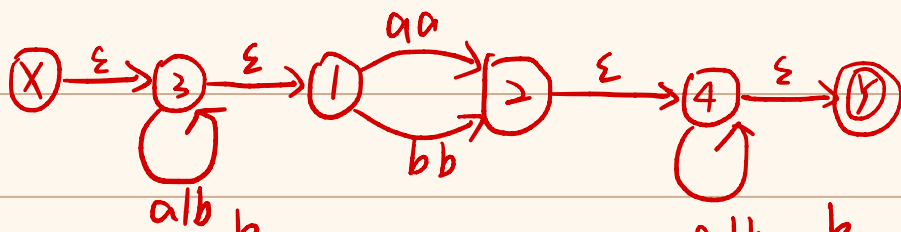
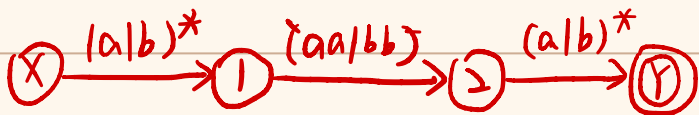
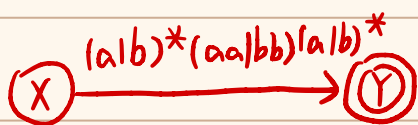


图 2-14 例 2.8 的 NFA M



例 2.8 正规表达式 $(a \mid b)^*(aa \mid bb)(a \mid b)^*$ 的 NFA M 如图 2-14 所示，试将其确定化为 DFA M'。

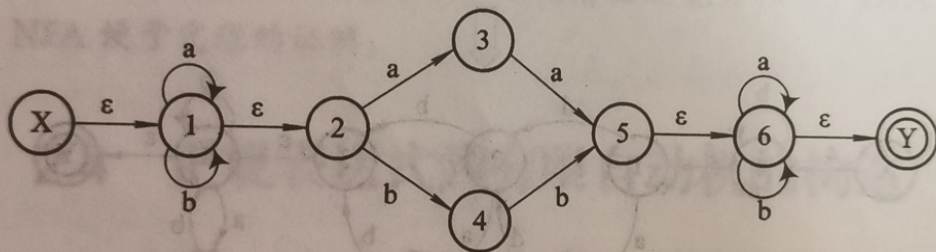


图 2-14 例 2.8 的 NFA M

表 2.4 例 2.8 的转换表

都经过串 1 (合并)					
I_a			I_b		
$\{X, 1, 2\}$	0	$\{1, 2, 3\}$	1	$\{1, 2, 4\}$	2
$\{1, 2, 3\}$	1	$\{1, 2, 3, 5, 6, Y\}$	3	$\{1, 2, 4\}$	2
$\{1, 2, 4\}$	2	$\{1, 2, 3\}$	1	$\{1, 2, 4, 5, 6, Y\}$	4
$\{1, 2, 3, 5, 6, Y\}$	3	$\{1, 2, 3, 5, 6, Y\}$	3	$\{1, 2, 4, 6, Y\}$	5
$\{1, 2, 4, 5, 6, Y\}$	4	$\{1, 2, 3, 6, Y\}$	6	$\{1, 2, 4, 5, 6, Y\}$	4
$\{1, 2, 4, 6, Y\}$	5	$\{1, 2, 3, 6, Y\}$	6	$\{1, 2, 4, 5, 6, Y\}$	4
$\{1, 2, 3, 6, Y\}$	6	$\{1, 2, 3, 5, 6, Y\}$	3	$\{1, 2, 4, 6, Y\}$	5

表 2.5 例 2.8 的状态转换矩阵

S	a	b
0	1	2
1	3	2
2	1	4
3	3	5
4	6	4
5	6	4
6	3	5

$$\{0, 1, 2\} a = \{1, 3\}$$

$$\{0, 2\} b = \{2, 4\}$$

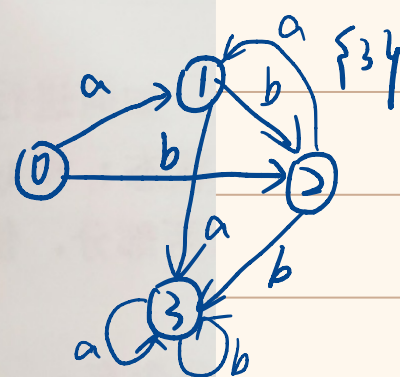
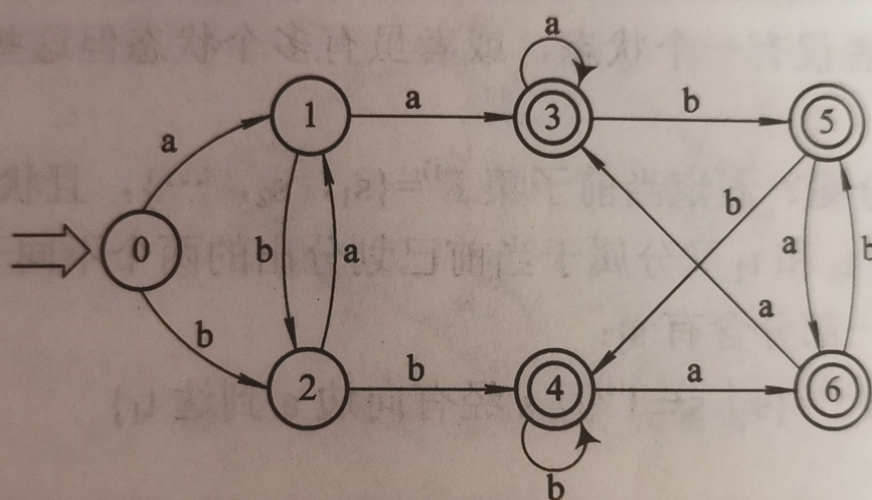
$$\{1\} a = \{3\}$$

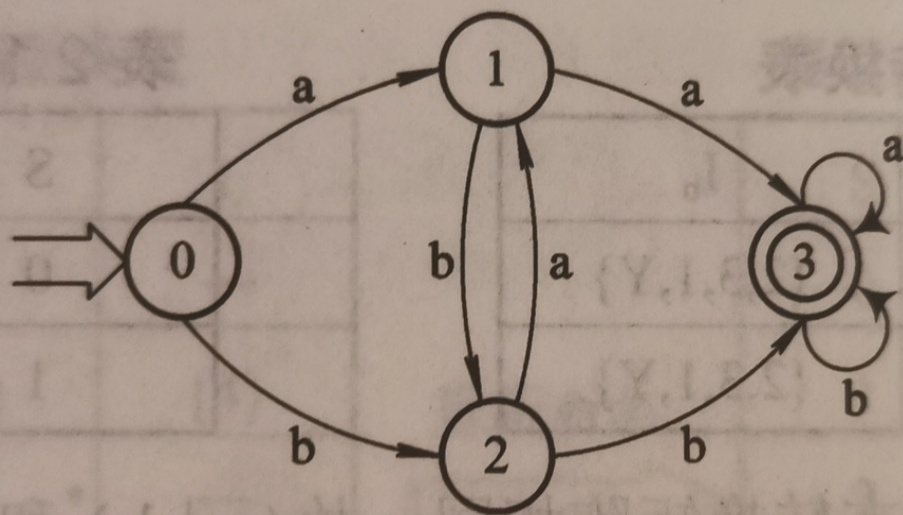
$$\{3, 4, 5, 6\} a = \{3, 6\}$$

$$\{3, 4, 5, 6\} b = \{4, 5\}$$

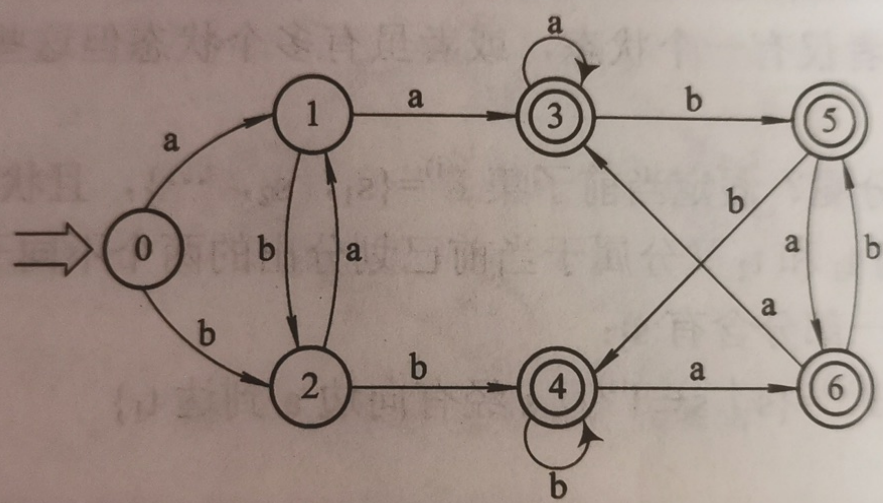
$$\{0\} a = \{1\}$$

$$\{0\} b = \{2\}$$





仅简.



第三章 语法分析

词法：正规文法

语法：上下文无关文法

语义：上下文有关文法

自顶向下分析法：从文法的开始符号出发，根据文法规则进行推导，最终推导出给定的句子来

自底向上分析法：从给定的输入串开始，根据文法规则逐步进行归约，直至归约到文法的开始符号为止

字母表的运算

- ① 乘积 (分配律) $\{0,1\}\{a,b\} = \{0a, 0b, 1a, 1b\}$
- ② n 次幂 $\begin{cases} \Sigma^0 = \{\Sigma\} \\ \Sigma^n = \Sigma^{n-1} \cdot \Sigma \end{cases}$
- ③ 连接 eg: $X = aqa, Y = bbb, xy = aqabbbb$
 $S = aab \quad S^3 = SSS = aabaabaab$

4. 非终结符和终结符

终结符	① 小写字母 ($a, b, c \dots$)	非终结符	① 大写字母
	② 运算符 ($+, -, *, /$)		② S 只代表开始符号
	③ 标点符号 ($(,), \dots$)		③ $expr$
	④ 数字 ($0 \sim 9$)		④ 程序结构的大写字母 E T F 项

包含空串的克林闭包

5. 句型 $S \xRightarrow{*} \alpha$ 由文法的开始符 S 出发，经过 0 步或有限步推导出来的符号串 α

句子 $S \xRightarrow{+} \alpha$ 由文法的开始符 S 出发，经过 1 步或有限步推导出来的符号串 α

b. 形式语言分类	0型文法	图灵机
	1型文法或上下文有关文法	线性界限自动机
	2型文法或上下文无关文法	下推自动机
	3型文法、正规文法或右线性文法	有限自动机

3型语言 $\subset$ 2型语言 $\subset$ 1型语言 $\subset$ 0型语言 $\times$ 不成立

7. 语法树 —— 短语、直接短语(简单短语)、素短语、句柄

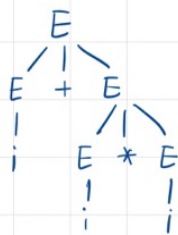
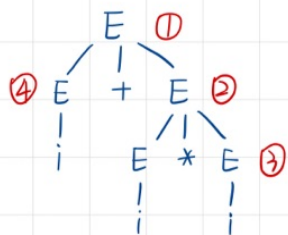
1. 怎么画语法树? (根据文法)

eg: 文法 $G[E]: E \rightarrow E+E \mid E \times E \mid (E) \mid i$, 字符串为: $i+i \times i$

两种 { 最左推导: 从左往右
最右推导: 从右往左

$E \rightarrow E+E \rightarrow i+E \rightarrow i+E \times E \rightarrow i+i \times E \rightarrow i+i \times i$

$E \rightarrow E+E \rightarrow E+E \times E \rightarrow E+E \times i \rightarrow E+i \times i \rightarrow i+i \times i$



不论哪种推导
得出的语法树都相同。

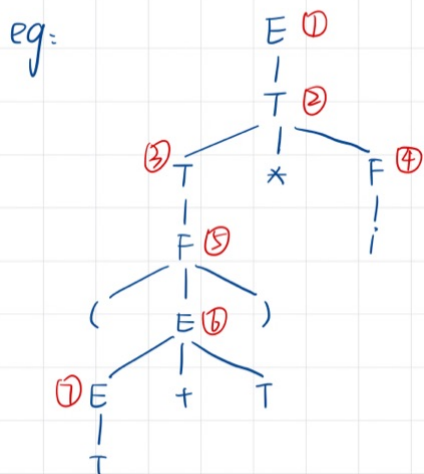
2. 短语有几个? 即找内部结点 能长叶子的

短语: ① $i+i \times i$ ② $i \times i$ ③④ i

直接短语: (直接长成的) i

素短语: (在短语中找, 至少含有一个终结符, 且不含其他更小素短语) i

句柄 (最左最小的树对应的符号串) i
在直接短语中



短语: ①② $(T+T) \times i$ ③⑤ $(T+T)$ ④ i ⑥ $T+T$ ⑦ T

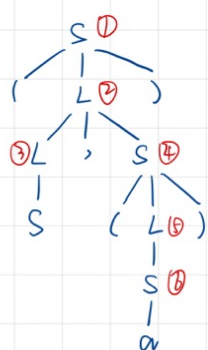
直接短语: i, T

素短语: $i, T+T$

句柄: T

eg: 对于文法 $G[S]: S \rightarrow (L) | aS | a$
 $L \rightarrow L, S | S$

- (1) 画句型 $(S, (a))$ 的语法树
 (2) 求短语、直接短语、句柄、素短语



短语: ① $(S, (a))$ ② $S, (a)$ ③ S ④ (a) ⑤ a
 直接短语: S, a
 句柄: S
 素短语: a

5. 二义性 (有两棵不同的语法树)

最左推导 } 树不同
 最右推导

8. 消除回溯、消除左递归

回溯产生的原因: 候选式存在公共的左因子

消除左递归

形式: $A \rightarrow A\alpha | \beta$ (α, β 为任意符号串)

改为: $\begin{cases} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' | \epsilon \end{cases}$

eg: $G[E]: E \rightarrow E+T | T$
 $T \rightarrow T * F | F$
 $F \rightarrow (E) | i$
 $\Rightarrow \begin{cases} E \rightarrow TE' \\ E' \rightarrow +TE' | \epsilon \end{cases} \quad \begin{cases} T \rightarrow FT' \\ T' \rightarrow *FT' | \epsilon \end{cases} \quad F \rightarrow (E) | i$

eg: $E \rightarrow E+T | E-T | T \Rightarrow \begin{cases} E \rightarrow TE' \\ E' \rightarrow +TE' | -TE' | \epsilon \end{cases}$

消除回溯

形式: $A \rightarrow \delta\beta_1 | \delta\beta_2 | \dots | \delta\beta_i | \beta_{i+1} | \dots | \beta_j$

改为: $\begin{cases} A \rightarrow \delta A' | \beta_{i+1} | \beta_j \\ A' \rightarrow \beta_1 | \dots | \beta_i \end{cases}$

eg: $G[A]: A \rightarrow aAB | a | b \Rightarrow \begin{cases} G'[A]: A \rightarrow aA' | b \\ A' \rightarrow AB | \epsilon \end{cases}$

9. LL(1)分析法 (又称预测分析法, 是一种不带回溯的非递归自顶向下分析表, 无二义性)

组成: 一张 LL(1) 分析表 (也称预测分析表)、一个先进后出分析栈、一个控制程序

11. 自底向上分析表的关键问题 —— 寻找可归约串

12. 规范推导和规范归约为逆过程，规范归约的(实质)：在移进过程中，当发现栈顶呈现

10. 构造 FIRST集合 FOLLOW集

句柄时，就用相应产生式的

左部符号进行替换

FIRST(A)是以A的开始符号的集合，A的所有可能推导的开头终结符或者是 ϵ

例 1. 后面跟的不是非终结符

$A \rightarrow aB \mid \epsilon$

$A \rightarrow c$

$\text{First}(A) = \{a, \epsilon, c\}$

2. 后面跟非终结符(-)

$A \rightarrow Ba$

$B \rightarrow b$

$\text{First}(A) = \{b\}$

3. 后面跟非终结符(=)

$A \rightarrow Bc$

$B \rightarrow b \mid \epsilon$

$\text{First}(A) = \{b, c, \epsilon\}$

4. 后面跟非终结符(三)

$A \rightarrow BC$

$B \rightarrow b \mid \epsilon$

$C \rightarrow c \mid \epsilon$

$\text{First}(A) = \{b, c, \epsilon\}$

Follow(A)为非终结符A后跟符号的集合，Follow(A)是所有句型中出现在紧接A之后的终结符或#

<1> 对于文法的开始符号S，置#于Follow(S)中；

<2> 若 $A \rightarrow \alpha B \beta$ 是一个产生式，则把 $\text{First}(\beta) \setminus \{\epsilon\}$ 加入到 Follow(B) 中

α 可以是：终结符 / 非终结符 / 空， β 可以是：终结符 / 非终结符，B后面要有东西

<3> 若 $A \rightarrow \alpha B$ 是一个产生式，或 $A \rightarrow \alpha B \beta$ 是一个产生式且 $\beta \rightarrow \epsilon$ ，则把 Follow(A) 加入到 Follow(B) 中

例：

$A \rightarrow aBC$

$A \rightarrow aBd$

$A \rightarrow BC$

$A \rightarrow Bd$

例: $G[S]: S \rightarrow aH$
 $H \rightarrow aMd$
 $H \rightarrow d$
 $M \rightarrow Ab$
 $M \rightarrow \varepsilon$
 $A \rightarrow aM$
 $A \rightarrow e$

First集: $First(S) = \{a\}$
 $First(H) = \{a, d\}$
 $First(M) = \{a, e, \varepsilon\}$
 $First(A) = \{a, e\}$

Follow集: $Follow(S) = \{\#\}$
 $Follow(H) = \{\#\}$
 $Follow(M) = \{d, b\}$
 $Follow(A) = \{b\}$

例: $G(E): E \rightarrow TE'$
 $E' \rightarrow +E | \varepsilon$
 $T \rightarrow FT'$
 $T' \rightarrow T | \varepsilon$
 $F \rightarrow PF'$
 $F' \rightarrow *F' | \varepsilon$
 $P \rightarrow (E) | a | b | \wedge$

First集: $First(E) = \{ (, a, b, \wedge \}$
 $First(E') = \{ +, \varepsilon \}$
 $First(T) = \{ (, a, b, \wedge \}$
 $First(T') = \{ (, a, b, \wedge, \varepsilon \}$
 $First(F) = \{ (, a, b, \wedge \}$
 $First(F') = \{ *, \varepsilon \}$
 $First(P) = \{ (, a, b, \wedge \}$

Follow集: $Follow(E) = \{ \#,) \}$
 $Follow(E') = \{ \#,) \}$
 $Follow(T) = \{ +, \#,) \}$
 $Follow(T') = \{ +, \#,) \}$
 $Follow(F) = \{ (, a, b, \wedge, +, \#,) \}$
 $Follow(F') = \{ (, a, b, \wedge, +, \#,) \}$
 $Follow(P) = \{ *, (, a, b, \wedge, +, \#,) \}$

<3.10> 已知文法 $G(A): A \rightarrow aAB | a$
 $B \rightarrow Bb | d$

<1> 试给出与 $G(A)$ 等价的 LL(1) 文法 $G'(A)$;

① 消除左递归: $B \rightarrow dB'$
 $B' \rightarrow bB' | \varepsilon$

② 消除回溯: $A \rightarrow aA'$
 $A' \rightarrow AB | \varepsilon$

改造后的文法为 $G'(A): A \rightarrow aA'$ First集: $FIRST(A) = \{a\}$ Follow集: $FOLLOW(A) = \{\#, d\}$
 $A' \rightarrow AB | \varepsilon$ $FIRST(A') = \{a, \varepsilon\}$ $FOLLOW(A') = \{\#, d\}$
 $B \rightarrow dB'$ $FIRST(B) = \{d\}$ $FOLLOW(B) = \{ | \}$
 $B' \rightarrow bB' | \varepsilon$ $FIRST(B') = \{b, \varepsilon\}$ $FOLLOW(B') = \{ | \}$

对于 $A' \rightarrow AB$ 来说, $FIRST(A) \cap FOLLOW(A') = \{a\} \cap \{\#, d\} = \emptyset$, 所以 $G'(A)$ 是等价的 LL(1) 的文法

<2> 构造 $G'(A)$ 的 LL(1) 分析表;

	a	b		d	#
A	$A \rightarrow aA'$				
A'	$A' \rightarrow AB$			$A' \rightarrow \varepsilon$	$A' \rightarrow \varepsilon$
B				$B \rightarrow dB'$	
B'		$B' \rightarrow bB'$	$B' \rightarrow \varepsilon$		

<3> 给出输入串 $aad1\#$ 的分析过程

符号栈	当前输入符号	输入串
#A	a	ad1#
#A'a	a	ad1#
#A'	a	d1#
#1BA	a	d1#
#1BA'a	a	d1#
#1BA'	d	1#
#1B	d	1#
#1B'd	d	1#
#1B'	1	#
#1	1	#
#	#	

13. 会构造算符优先分析表: 求 FIRSTVT 集和 LASTVT 集, 用关系图法求优先函数

算符优先文法

优先关系表

<1> $a \equiv b$, 对形如 $P \rightarrow \dots ab \dots$ 或 $P \rightarrow \dots aRb \dots$ 的产生式

<2> $a < b$, 对形如 $P \rightarrow \dots aR \dots$ 的产生式, 有 $b \in \text{FIRSTVT}(R)$ 横看

<3> $a > b$ 对形如 $P \rightarrow \dots Rb \dots$ 的产生式, 有 $a \in \text{LASTVT}(R)$ 竖看

eg: $G[E]: E \rightarrow E+T \mid T$
 $T \rightarrow T*F \mid F$
 $F \rightarrow (E) \mid i$

$\text{FIRSTVT}(E) = \{+, *, (, i\}$

$\text{LASTVT}(E) = \{+, *,), i\}$

$\text{FIRSTVT}(T) = \{*, (, i\}$

$\text{LASTVT}(T) = \{*,), i\}$

$\text{FIRSTVT}(F) = \{(, i\}$

$\text{LASTVT}(F) = \{), i\}$

$a < b$

	+	*	(	)	#
+	>	<	<	<	>
*	>	>	<	<	>
(	>	>			>
)	<	<	<	<	=
#	>	>			>

#E# 得 #=#

< FIRSTVT(E)

LASTVT(E) > #

> 竖 → 横
f g

f

<3.12> 对文法 $G[E]$:
 $E \rightarrow E+T \mid T$
 $T \rightarrow T*P \mid P$
 $P \rightarrow i$

(1) 构造该文法的优先关系表 (不考虑括号 #), 并指出此文法是否为算符优先文法;

FIRSTVT 集: $FIRSTVT(E) = \{+, *, i\}$
 $FIRSTVT(T) = \{*, i\}$
 $FIRSTVT(P) = \{i\}$

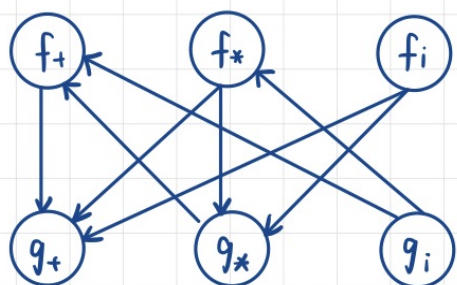
LASTVT 集: $LASTVT(E) = \{+, *, i\}$
 $LASTVT(T) = \{*, i\}$
 $LASTVT(P) = \{i\}$

则可得

	+	*	i
+	>	<	<
*	>	>	<
i	>	>	

> 竖 + $\rightarrow$ 横

(2) 构造文法 $G[E]$ 的优先函数



	+	*	i
f	2	4	4
g	1	3	5

14. 算符优先分析法的关键问题是 —— 寻找最左素短语.

15. LR分析法的定义, 四个操作动作 (移进, 归约, 接受, 报错)

定义 是一种自底向上进行规范规约的语法分析方法

实质 一个带分析栈的 DFA

16. 活前缀定义, 项目定义, 会识别四种项目 (移进, 归约, 接受, 特约)

规范句型的一个前缀 (不含句柄之后的任何符号)

文法 $G[S]$ 中每一个产生式的右部添加一个圆点 "•"

• 后面是终结符

① 归约项目 ($A \rightarrow \alpha \cdot$) • 在最后

③ 移进项目 ($A \rightarrow \alpha \cdot x \beta$)

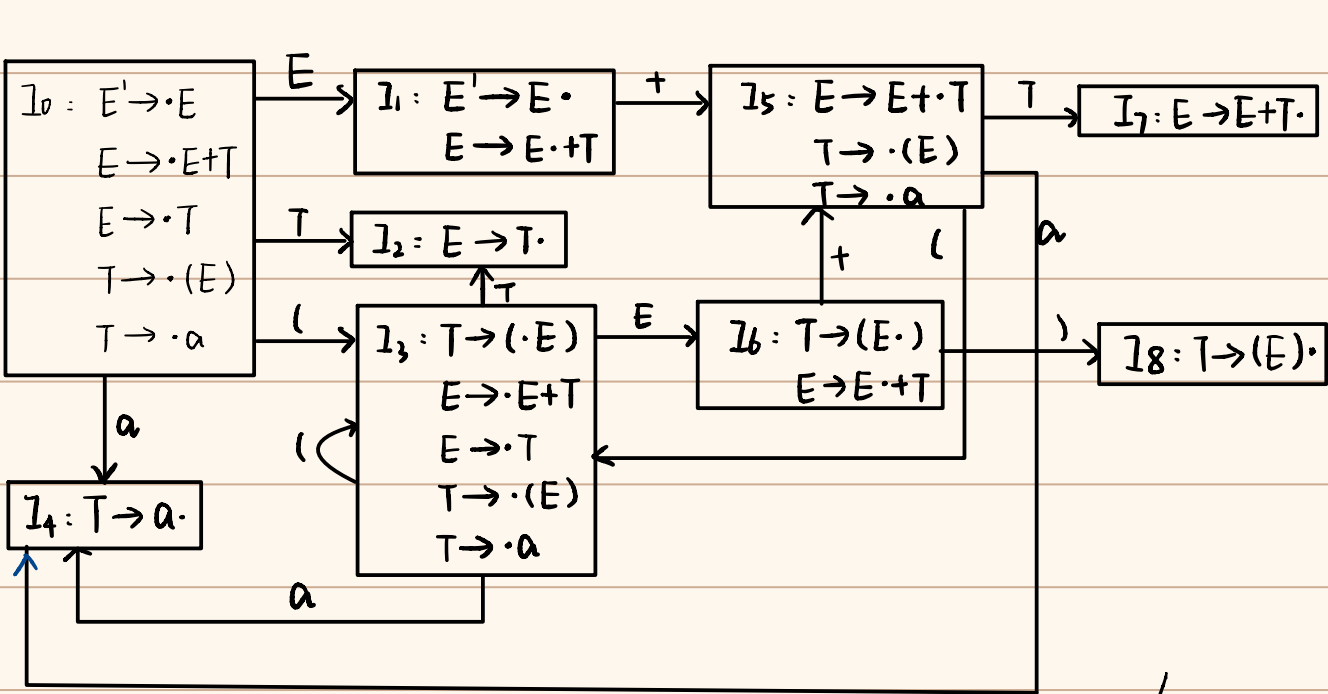
② 接受项目 ($S \rightarrow \alpha \cdot$) 开始文法对应的

④ 特约项目 ($A \rightarrow \alpha \cdot x \beta$) • 后面是非终结符

17. 会构造 LR(0) 分析表：拓广文法 → 列项目 → 项目集规范簇 → 分析表

<3.19> 试构造下述文法的 LR(0) 分析表：
 $G[E]: E \rightarrow E+T \mid T$
 $T \rightarrow (E) \mid a$

由文法可得 $G[E']$ ：
(0) $E' \rightarrow E$
(1) $E \rightarrow E+T$
(2) $E \rightarrow T$
(3) $T \rightarrow (E)$
(4) $T \rightarrow a$



状态	ACTION					GOTO	
	+	(	)	a	#	E	T
0		S ₃		S ₄		1	2
1	S ₅				acc		
2	r ₂	r ₂	r ₂	r ₂	r ₂		
3		S ₃		S ₄		6	2
4	r ₄	r ₄	r ₄	r ₄	r ₄		
5		S ₃		S ₄			7
6	S ₅		S ₈				
7	r ₁	r ₁	r ₁	r ₁	r ₁		
8	r ₃	r ₃	r ₃	r ₃	r ₃		

LR(0)

18. 会构造 SLR(1) 分析表, 步骤同上, 只是多求一步 FOLLOW 集

(移进 — 归约) (归约 — 归约) 同一个项目里有冲突
• 后面是终结符 • 在最后

第四章

1. 语义检查: 动态(运行)和静态(编译), 静态所检查的几个方面

类型检查

控制流检查

一致性检查

2. 语法制导翻译的定义: 为每个产生式配上一个翻译子程序, 并在语法分析的同时执行这些子程序

自顶向下 自底向上

3. 文法符号的属性: 继承属性和综合属性

4. 会将表达式用逆波兰方式(后缀)表示

5. 会写出代码的四元式(控制语句: if else, for, do while, while)

if ($m < n$) $n = 0$;

else $n = 1$;

1 ($j <, m, n, 3$)

2 ($j, _, _, 5$)

3 ($=, 0, _, n$)

4 ($j, _, _, 6$)

5 ($=, 1, _, n$)

6

3.1 完成下列选择题:

- (1) 程序语言的语义需要_____用来描述。
A. 上下文无关文法
B. 上下文有关文法
C. 正规文法
D. 短语文法
- (2) 2型文法对应_____。
A. 图灵机
B. 有限自动机
C. 下推自动机
D. 线性界限自动机
- (3) 下述结论中, _____是正确的。
A. 1型语言 $\subset$ 0型语言
B. 2型语言 $\subset$ 1型语言
C. 3型语言 $\subset$ 2型语言
D. A~C均不成立
- (4) 有限状态自动机能识别_____。
A. 上下文无关文法
B. 上下文有关文法
C. 正规文法
D. 短语文法
- (5) 文法 $G[S]: S \rightarrow xSx \mid y$ 所识别的语言是_____。
A. xyx
B. $(xyx)^*$
C. $x^n y x^n (n \geq 0)$
D. $x^* y x^*$
- (6) 只含有单层分枝的子树称为“简单子树”, 则句柄的直观解释是_____。
A. 子树的末端结点(即树叶)组成的符号串
B. 简单子树的末端结点组成的符号串
C. 最左简单子树的末端结点组成的符号串
D. 最左简单子树的末端结点组成的符号串且该符号串必须含有终结符
- (7) 下面对语法树错误的描述是_____。
A. 根结点用文法 $G[S]$ 的开始符 S 标记
B. 每个结点用 $G[S]$ 的一个终结符或非终结符标记
C. 如果某结点标记为 ϵ , 则它必为叶结点
D. 内部结点可以是非终结符
- (8) 由文法开始符 S 经过零步或多步推导产生的符号序列是_____。
A. 短语
B. 句柄
C. 句型
D. 句子
- (9) 设文法 $G[S]: S \rightarrow SA \mid A$
 $A \rightarrow a \mid b$

则对句子 aba 的规范推导是_____。

- A. $S \Rightarrow SA \Rightarrow SAA \Rightarrow AAA \Rightarrow aAA \Rightarrow abA \Rightarrow aba$
B. $S \Rightarrow SA \Rightarrow SAA \Rightarrow AAA \Rightarrow AAa \Rightarrow Aba \Rightarrow aba$
C. $S \Rightarrow SA \Rightarrow SAA \Rightarrow SAa \Rightarrow Sba \Rightarrow Aba \Rightarrow aba$
D. $S \Rightarrow SA \Rightarrow Sa \Rightarrow SAa \Rightarrow Sba \Rightarrow Aba \Rightarrow aba$

(10) 如果文法 $G[S]$ 是无二义的, 则它的任何句子 α 其_____。

- A. 最左推导和最右推导对应的语法树必定相同
- B. 最左推导和最右推导对应的语法树可能不同
- C. 最左推导和最右推导必定相同
- D. 可能存在两个不同的最右推导, 但它们对应的语法树相同

(11) 一个句型的分析树代表了该句型的_____。

- A. 推导过程
- B. 归约过程
- C. 生成过程
- D. 翻译过程

(12) 规范归约中的“可归约串”由_____定义。

- A. 直接短语
- B. 最右直接短语
- C. 最左直接短语
- D. 最左素短语

(13) 规范归约是指_____。

- A. 最左推导的逆过程
- B. 最右推导的逆过程
- C. 规范推导
- D. 最左归约的逆过程

(14) 文法 $G[S]$: $S \rightarrow aAcB \mid Bd$

$A \rightarrow AaB \mid c$

$B \rightarrow bScA \mid b$

则句型 $aAcBdccc$ 的短语是_____。

- A. Bd
- B. cc
- C. a
- D. b

(15) 文法 $G[E]$: $E \rightarrow E+T \mid T$

$T \rightarrow T*P \mid P$

$P \rightarrow (E) \mid i$

则句型 $P+T+i$ 的句柄和最左素短语是_____。

- A. $P+T$ 和 T
- B. P 和 $P+T$
- C. i 和 $P+T+i$
- D. P 和 P

(16) 采用自顶向下分析, 必须_____。

- A. 消除左递归
- B. 消除右递归
- C. 消除回溯
- D. 提取公共左因子

(17) 对文法 $G[E]$: $E \rightarrow E+S \mid S$

$S \rightarrow S*F \mid F$

$F \rightarrow (E) \mid i$

则 $FIRST(S) =$ _____。

- A. $\{()$
- B. $\{(, i)$
- C. $\{i\}$
- D. $\{(,)\}$

(18) 确定的自顶向下分析要求文法满足_____。

- A. 不含左递归
- B. 不含二义性
- C. 无回溯
- D. A~C 项

(19) 递归下降分析器由一组递归函数组成, 且每一个函数对应文法的_____。

- A. 一个终结符
- B. 一个非终结符
- C. 多个终结符
- D. 多个非终结符

(20) LL(1)分析表需要预先定义和构造两族与文法有关的集合_____。

- A. $FIRST$ 和 $FOLLOW$
- B. $FIRSTVT$ 和 $FOLLOW$
- C. $FIRST$ 和 $LASTVT$
- D. $FIRSTVT$ 和 $LASTVT$

(21) 设 a, b, c 是文法的终结符且满足优先关系 $a \preceq b$ 和 $b \preceq c$, 则_____。

- (32) 一个_____指明了在分析过程中某时刻能看到产生式多大一部分。
A. 活前缀 B. 前缀 C. 项目 D. 项目集
- (33) 若 B 为非终结符, 则 $A \rightarrow \alpha \cdot B \beta$ 为_____项目。
A. 接受 B. 归约 C. 移进 D. 待约
- (34) 在 LR(0) 的 ACTION 子表中, 如果某一行中存在标记为 " r_j " 的栏, 则_____。
A. 该行必定填满 r_j B. 该行未填满 r_j
C. 其他行也有 r_j D. GOTO 子表中也有 r_j
- (35) LR 分析法解决“移进/归约”冲突时, 左结合意味着_____。
A. 打断联系实行移进 B. 打断联系实行归约
C. 建立联系实行移进 D. 建立联系实行归约
- (36) LR 分析法解决“移进/归约”冲突时, 右结合意味着_____。
A. 打断联系实行归约 B. 建立联系实行归约
C. 建立联系实行移进 D. 打断联系实行移进
- (37) 若项目集 I_k 含有 $A \rightarrow \alpha \cdot$, 则在状态 k 时, 仅当面临的输入符号 $a \in FOLLOW(A)$ 才采取“将 α 归约为 A ”动作的一定是_____。
A. LALR(1) 文法 B. LR(0) 文法
C. LR(1) 文法 D. SLR(1) 文法
- (38) 同心集合并又可能产生新的_____冲突。
A. 归约/接受 B. 移进/移进
C. 移进/归约 D. 归约/归约
- 【解答】